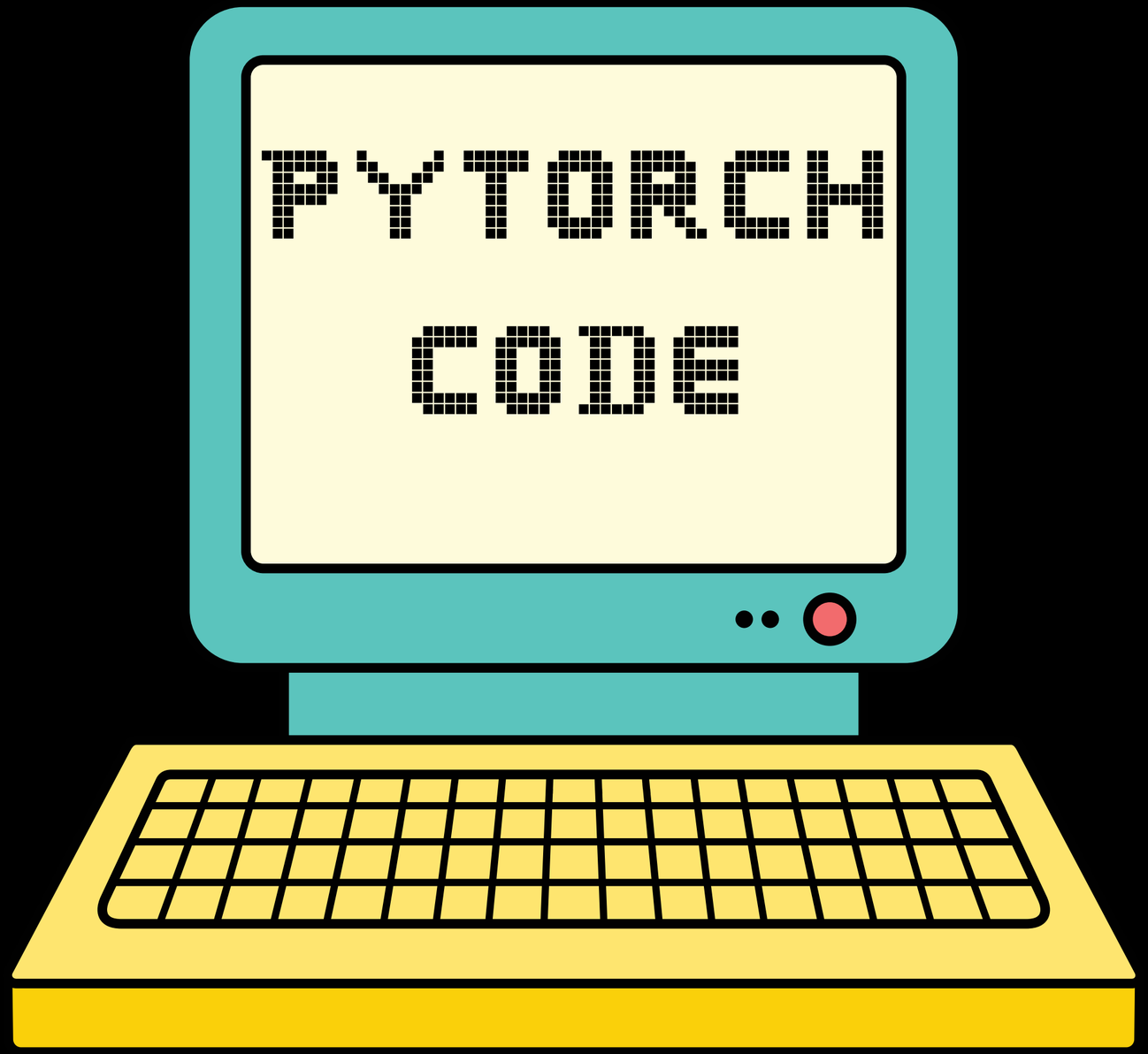


Start

DATA OVERVIEW

- 47,000+ Rows
- 18 columns
- Balanced Dataset
- Train-Test Split:
  - 80% train
  - 20% test

SNIPPET OF VIDEO GAME DATASET

	Game Title	User Rating	Age Group Targeted	Price	Platform	Requires Special Device	Developer	Publisher	Release Year	Genre	Multiplayer	Game Length (Hours)	Graphics Quality	Soundtrack Quality	Story Quality	User Review Text	Game Mode	Min Number of Players
0	Grand Theft Auto V	36.4	All Ages	41.41	PC	No	Game Freak	Innersloth	2015	Adventure	No	55.3	Medium	Average	Poor	Solid game, but too many bugs.	Offline	1
1	The Sims 4	38.3	Adults	57.56	PC	No	Nintendo	Electronic Arts	2015	Shooter	Yes	34.6	Low	Poor	Poor	Solid game, but too many bugs.	Offline	3
2	Minecraft	26.8	Teens	44.93	PC	Yes	Bungie	Capcom	2012	Adventure	Yes	13.9	Low	Good	Average	Great game, but the graphics could be better.	Offline	5

PREDITION FLOW

Raw Data → Data Cleaning → Embedding → Models → Prediction

1 2 3 4 5

1. WHAT'S THE GOAL?

1. WHAT'S THE GOAL?

Predict user ratings

DATA CLEANING

- **Predictive column**: User Reviews
- **What's used**: Price, game length, soundtrack story and graphic quality.
- **What's cut**: game title, user review text

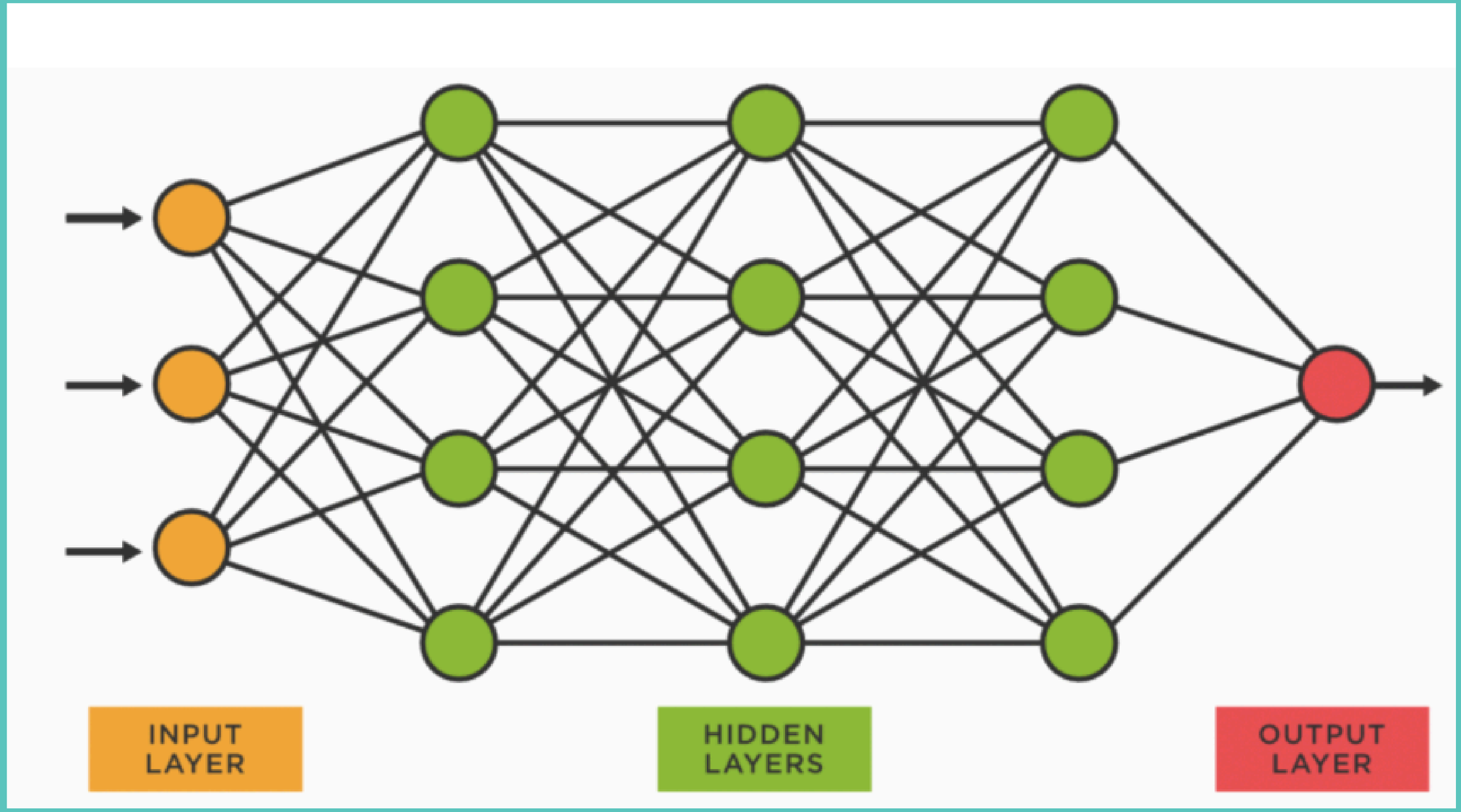
IDENTIFYING COLUMN GROUPS

- Target column:
  - User Rating (continuous output)
- Numerical columns:
  - Price
  - Release year
  - Game Length (Hours)
  - Min number of players
- Categorical columns:
  - Small categorical features:
    - Age Group Targeted
    - Requires Special Device
    - Multiplayer
    - Game Mode
  - Big Categorical Features:
    - Publisher
    - Genre
    - Platform
    - Developer

WHY EMBEDDING?

Neural Networks work best with numerical data, and categorical columns (like Publisher, Genre, etc.) need to be converted into numerical representations.

Embeddings turn large, categorical data (like Platform or Publisher) into smaller, more manageable numbers, while keeping important relationships between them. This helps make the model simpler and more efficient.


INPUT LAYER
HIDDEN LAYERS
OUTPUT LAYER

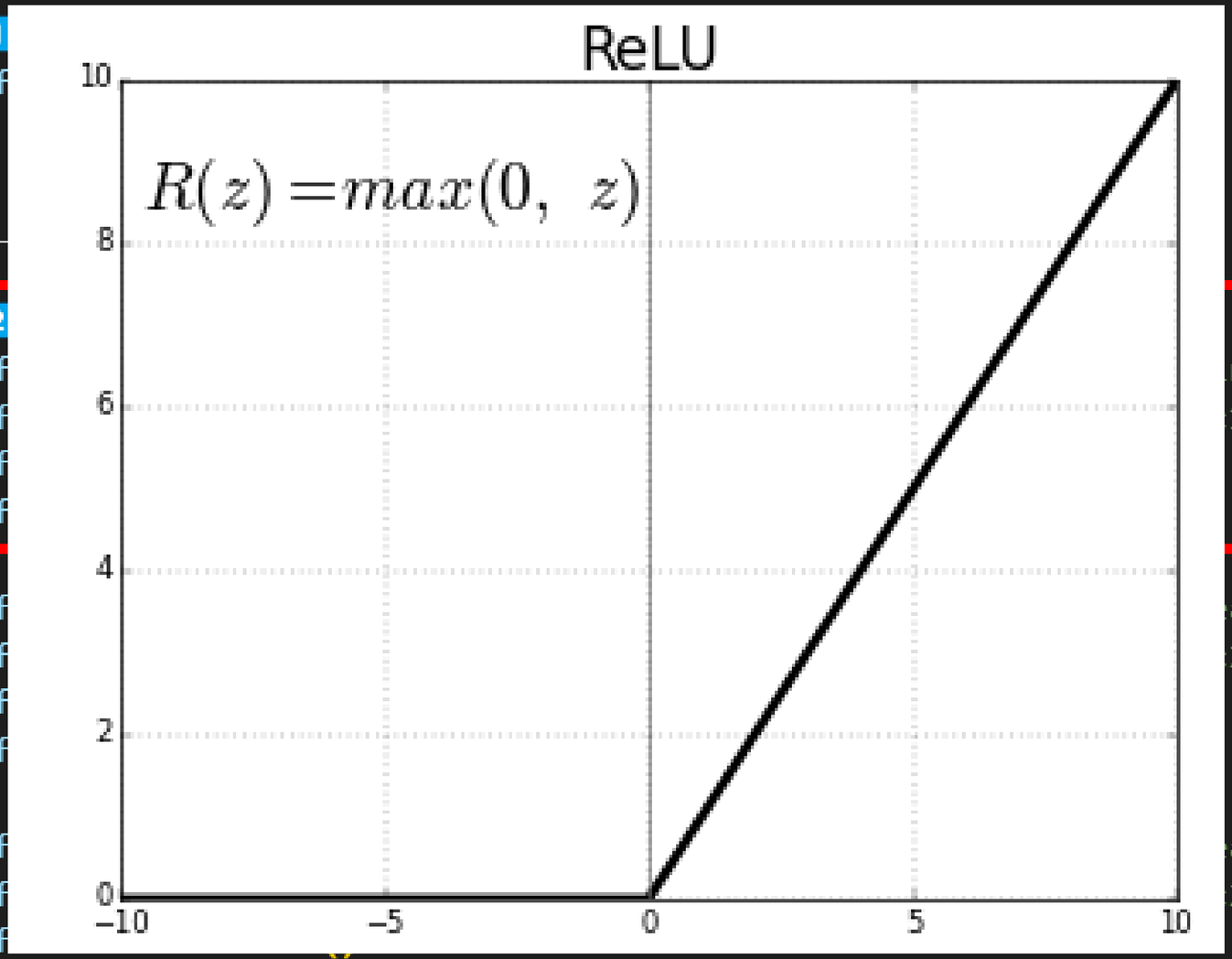
```python
[99] class TabularRegressor(nn.Module):
        def __init__(self, embedding_sizes, n_numeric, hidden_dims=[128, 64], p_dropout=0.1):
            super().__init__()

            # 1 Create embeddings for each categorical column
            self.embeddings = nn.ModuleList(
                [nn.Embedding(n_cat, dim) for n_cat, dim in embedding_sizes]
            )
            emb_out_dim = sum(dim for _, dim in embedding_sizes)

            # 2 Define layers
            self.fc1 = nn.Linear(emb_out_dim + n_numeric, hidden_dims[0])  # First linear layer
            self.bn1 = nn.BatchNorm1d(hidden_dims[0])  # BatchNorm layer for fc1
            self.relu1 = nn.ReLU()  # ReLU activation
            self.dropout1 = nn.Dropout(p_dropout)  # Dropout after fc1

            self.fc2 = nn.Linear(hidden_dims[0], hidden_dims[1])  # Second linear layer
            self.bn2 = nn.BatchNorm1d(hidden_dims[1])  # BatchNorm layer for fc2
            self.relu2 = nn.ReLU()  # ReLU activation
            self.dropout2 = nn.Dropout(p_dropout)  # Dropout after fc2

            self.fc3 = nn.Linear(hidden_dims[1], hidden_dims[1])  # Second linear layer
            self.bn3 = nn.BatchNorm1d(hidden_dims[1])  # BatchNorm layer for fc2
            self.relu3 = nn.ReLU()  # ReLU activation
            self.dropout3 = nn.Dropout(p_dropout)  # Dropout after fc2

            self.fc4 = nn.Linear(hidden_dims[1], 1)  # Output layer (regression)
```

KEY CODE

```python
[99] class TabularRegressor(nn.Module):
        def __init__(self, embedding_sizes, n_numeric, hidden_dims=[128, 64], p_dropout=0.1):
            super().__init__()

            # 1️⃣ Create embeddings for each categorical column
            self.embeddings = nn.ModuleList(
                [nn.Embedding(n_cat, dim) for n_cat, dim in embedding_sizes]
            )
            emb_out_dim = sum(dim for _, dim in embedding_sizes)

            # 2️⃣ Define layers
            self.fc1 = nn.Linear(emb_out_dim + n_numeric, hidden_dims[0])  # First linear layer
            self.bn1 = nn.BatchNorm1d(hidden_dims[0])  # BatchNorm layer for fc1
            self.relu1 = nn.ReLU()  # ReLU activation
            self.dropout1 = nn.Dropout(p_dropout)  # Dropout after fc1

            self.fc2 = nn.Linear(hidden_dims[0], hidden_dims[1])  # Second linear layer
            self.bn2 = nn.BatchNorm1d(hidden_dims[1])  # BatchNorm layer for fc2
            self.relu2 = nn.ReLU()  # ReLU activation
            self.dropout2 = nn.Dropout(p_dropout)  # Dropout after fc2

            self.fc3 = nn.Linear(hidden_dims[1], hidden_dims[1])  # Second linear layer
            self.bn3 = nn.BatchNorm1d(hidden_dims[1])  # BatchNorm layer for fc2
            self.relu3 = nn.ReLU()  # ReLU activation
            self.dropout3 = nn.Dropout(p_dropout)  # Dropout after fc2

            self.fc4 = nn.Linear(hidden_dims[1], 1)  # Output layer (regression)
```

KEY CODE

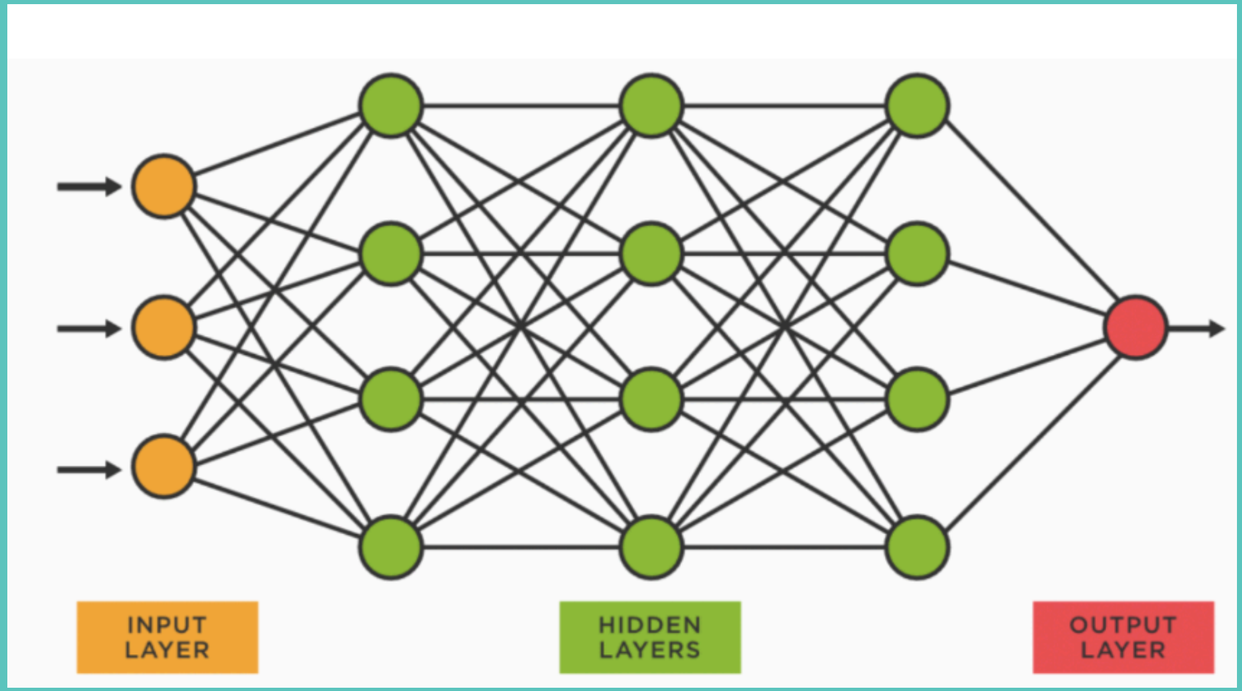
```
[99] class TabularRegressor(nn.Module):
        def __init__(self, embedding_sizes, n_numeric, hidden_dims=[128, 64], p_dropout=0.1):
            super().__init__()

            # 1
            self

            )
            emb_

            # 2
            self                                                    rst linear layer
            self                                                    1
            self
            self

            self                                                    ar layer
            self                                                    2
            self
            self

            self                                                    ar layer
            self                                                    2
            self
            self.dropout3 = nn.Dropout(p_dropout)  # Dropout after fc2

            self.fc4 = nn.Linear(hidden_dims[1], 1)  # Output layer (regression)
```



KEY CODE

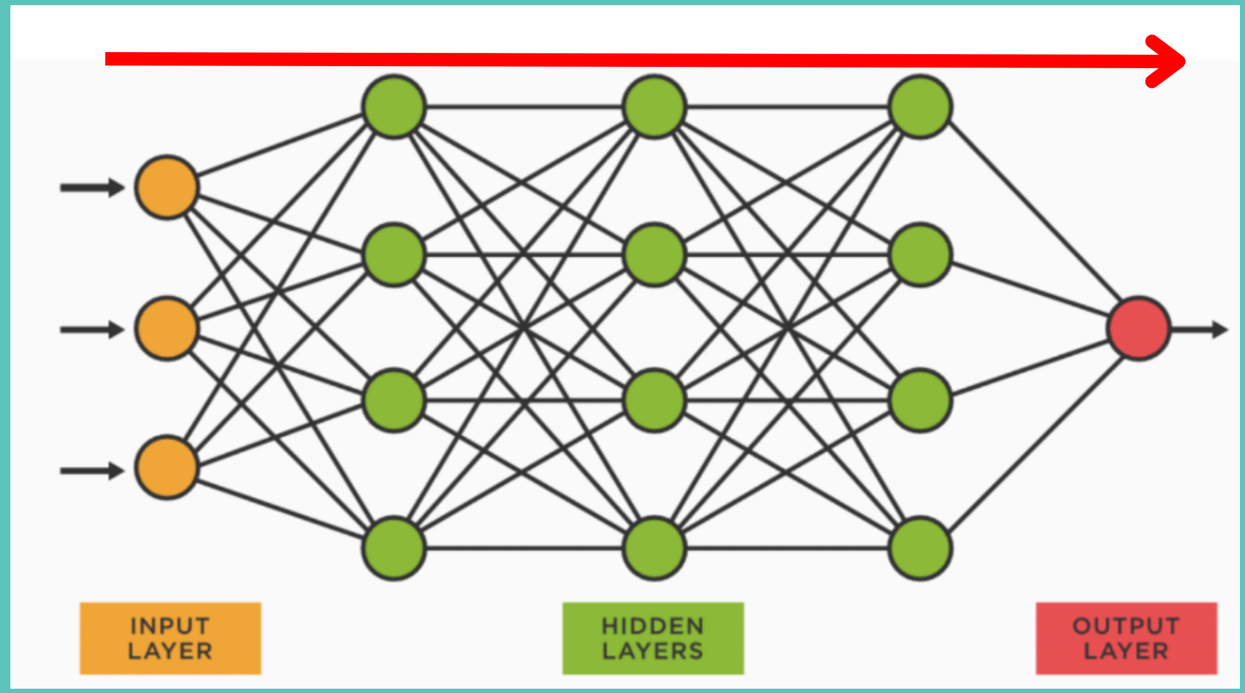
```python
def forward(self, x_cat, x_num):
    # Embedding lookup for categorical data
    embs = [emb(x_cat[:, i]) for i, emb in enumerate(self.embeddings)]
    x = torch.cat(embs + [x_num], dim=1)  # Concatenate embeddings and numerical data

    # Pass through the layers one by one
    x = self.fc1(x)
    x = self.bn1(x)
    x = self.relu1(x)
    x = self.dropout1(x)

    x = self.fc2(x)
    x = self.bn2(x)
    x = self.relu2(x)
    x = self.dropout2(x)

    x = self.fc3(x)
    x = self.bn3(x)
    x = self.relu3(x)
    x = self.dropout3(x)

    x = self.fc4(x)  # Final output
```

KEY CODE


INPUT LAYER
HIDDEN LAYERS
OUTPUT LAYER

EPOCHS

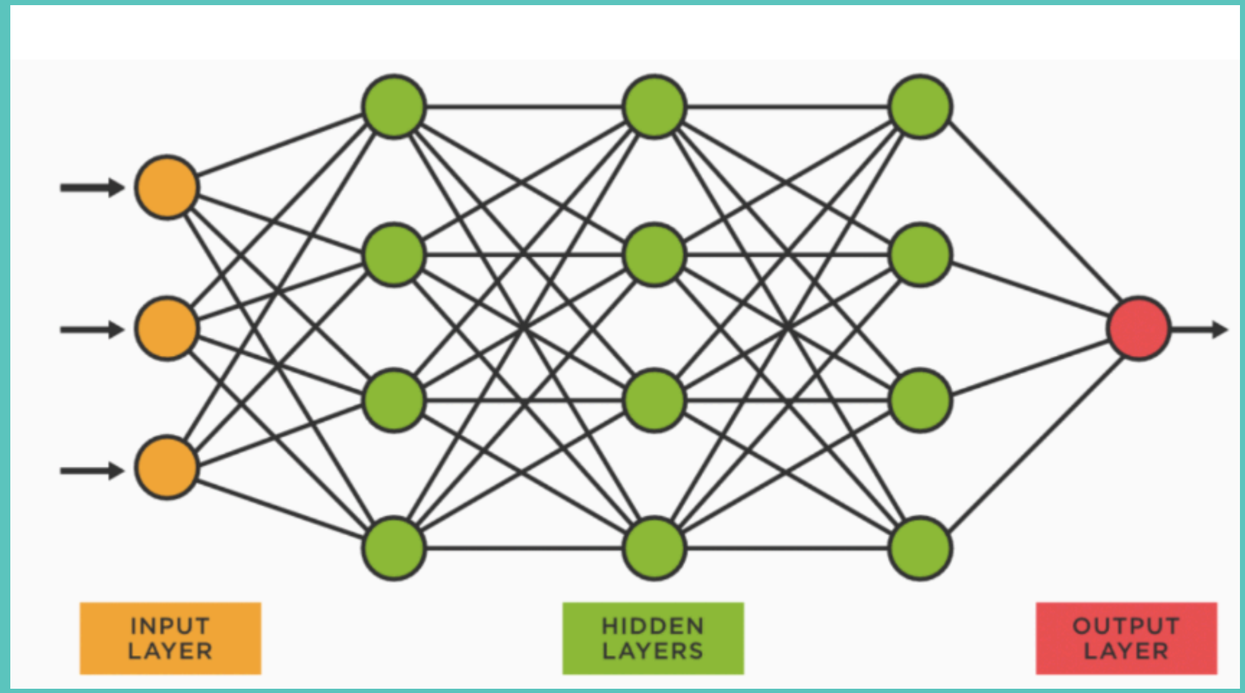
Iterate this many times




```
for epoch in range(30):
    model.train()
    for cats, nums, y in train_dl:
        cats, nums, y = cats.to(device), nums.to(device), y.to(device)
        preds = model(cats, nums)
        loss  = criterion(preds, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

EPOCHS

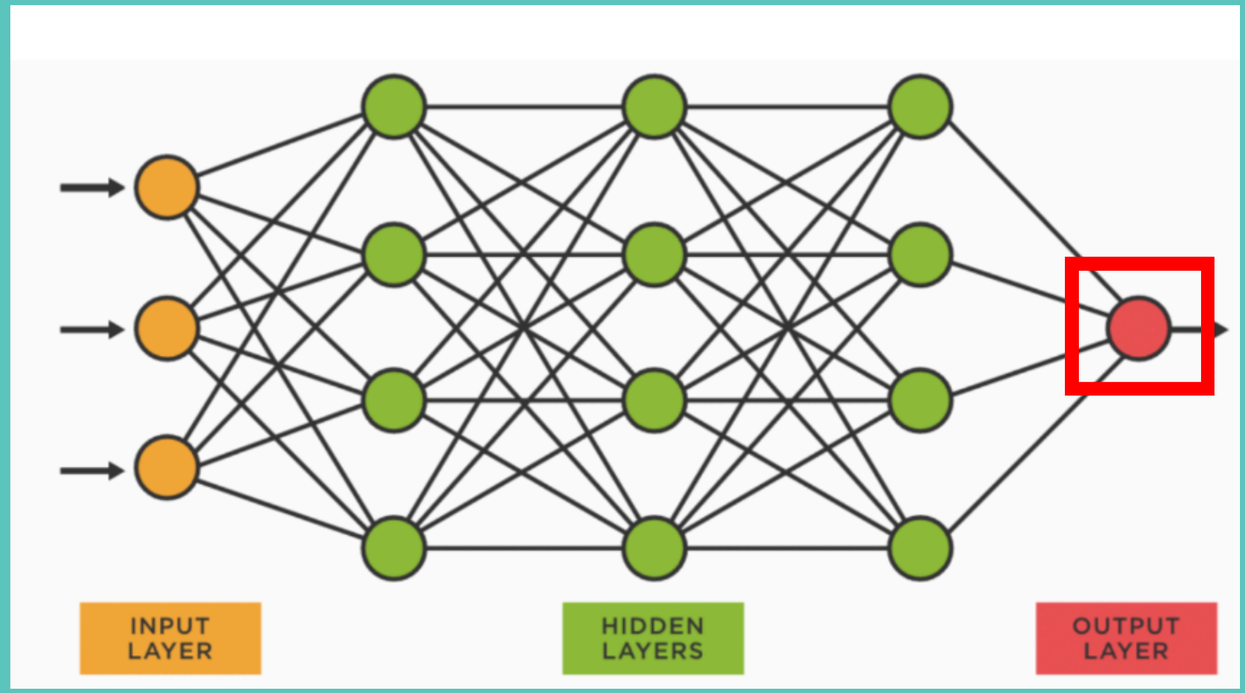
Bringing the data together




```
for epoch in range(30):
    model.train()
    for cats, nums, y in train_dl:
        cats, nums, y = cats.to(device), nums.to(device), y.to(device)
        preds = model(cats, nums)
        loss  = criterion(preds, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

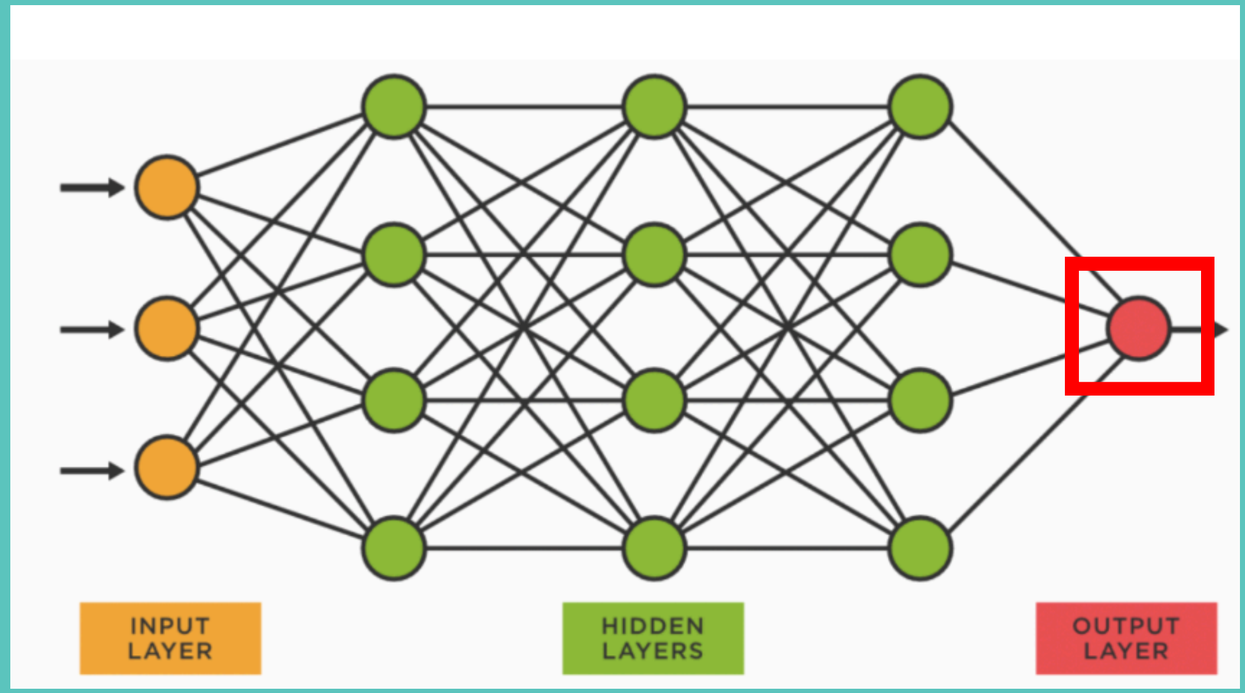
EPOCHS

Create Prediction




```
for epoch in range(30):
    model.train()
    for cats, nums, y in train_dl:
        cats, nums, y = cats.to(device), nums.to(device), y.to(device)
        preds = model(cats, nums)
        loss  = criterion(preds, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

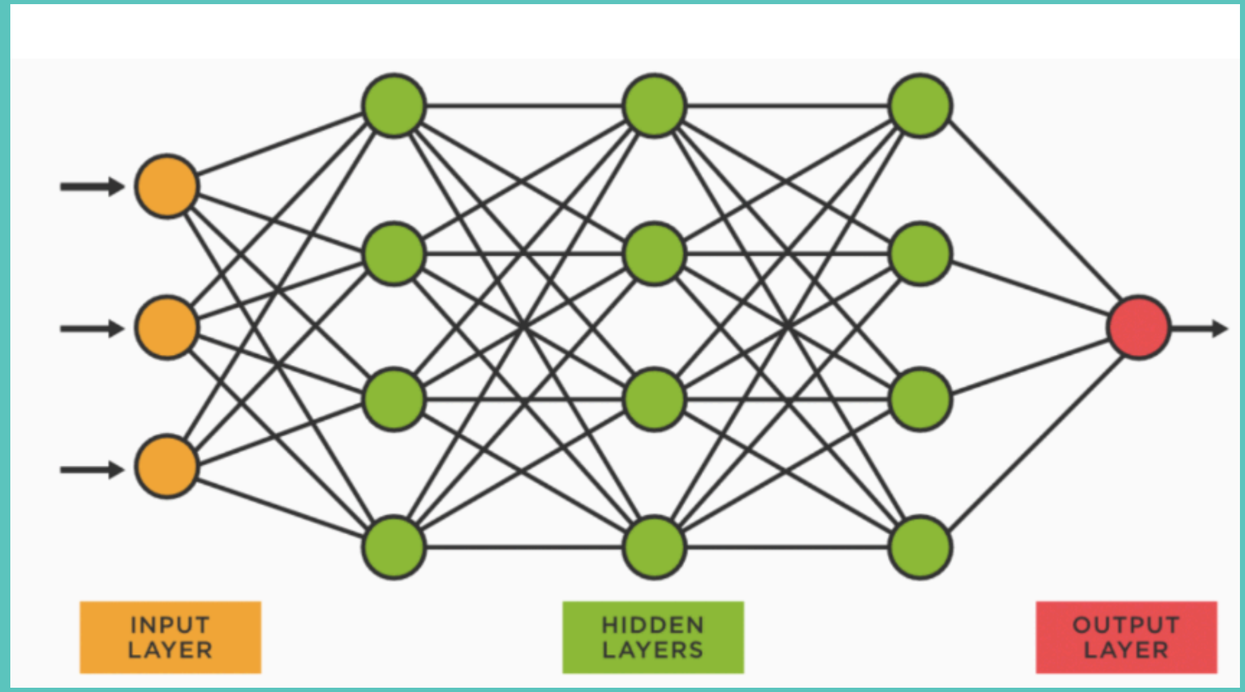
EPOCHS



Calculate Loss

```
for epoch in range(30):
    model.train()
    for cats, nums, y in train_dl:
        cats, nums, y = cats.to(device), nums.to(device), y.to(device)
        preds = model(cats, nums)
        loss  = criterion(preds, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

EPOCHS

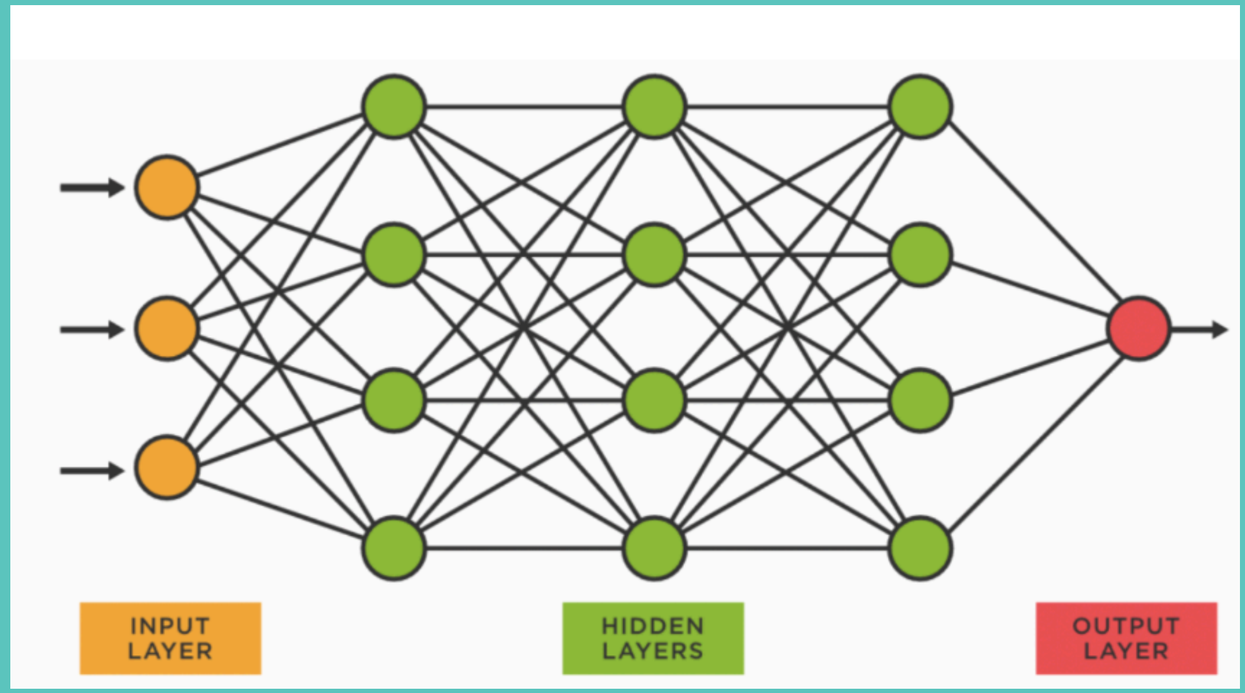


Optimize

```
for epoch in range(30):
    model.train()
    for cats, nums, y in train_dl:
        cats, nums, y = cats.to(device), nums.to(device), y.to(device)
        preds = model(cats, nums)
        loss  = criterion(preds, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

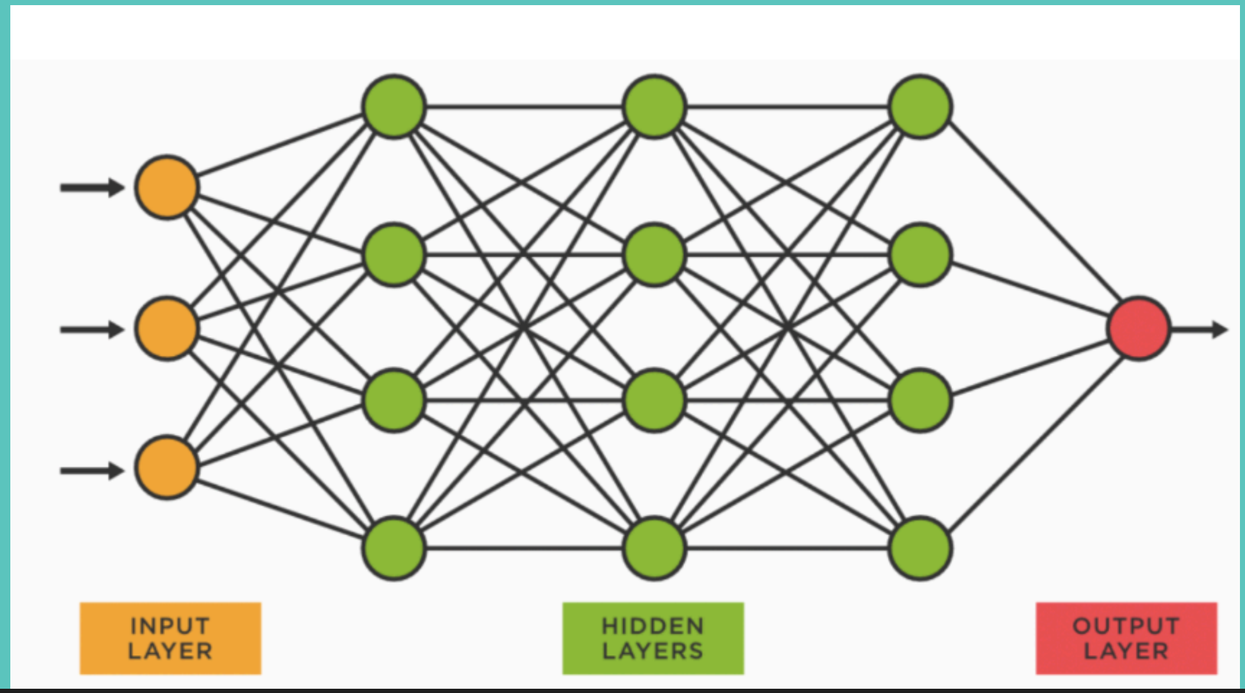
EPOCHS

Preforms backpropagation




```
for epoch in range(30):
    model.train()
    for cats, nums, y in train_dl:
        cats, nums, y = cats.to(device), nums.to(device), y.to(device)
        preds = model(cats, nums)
        loss  = criterion(preds, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

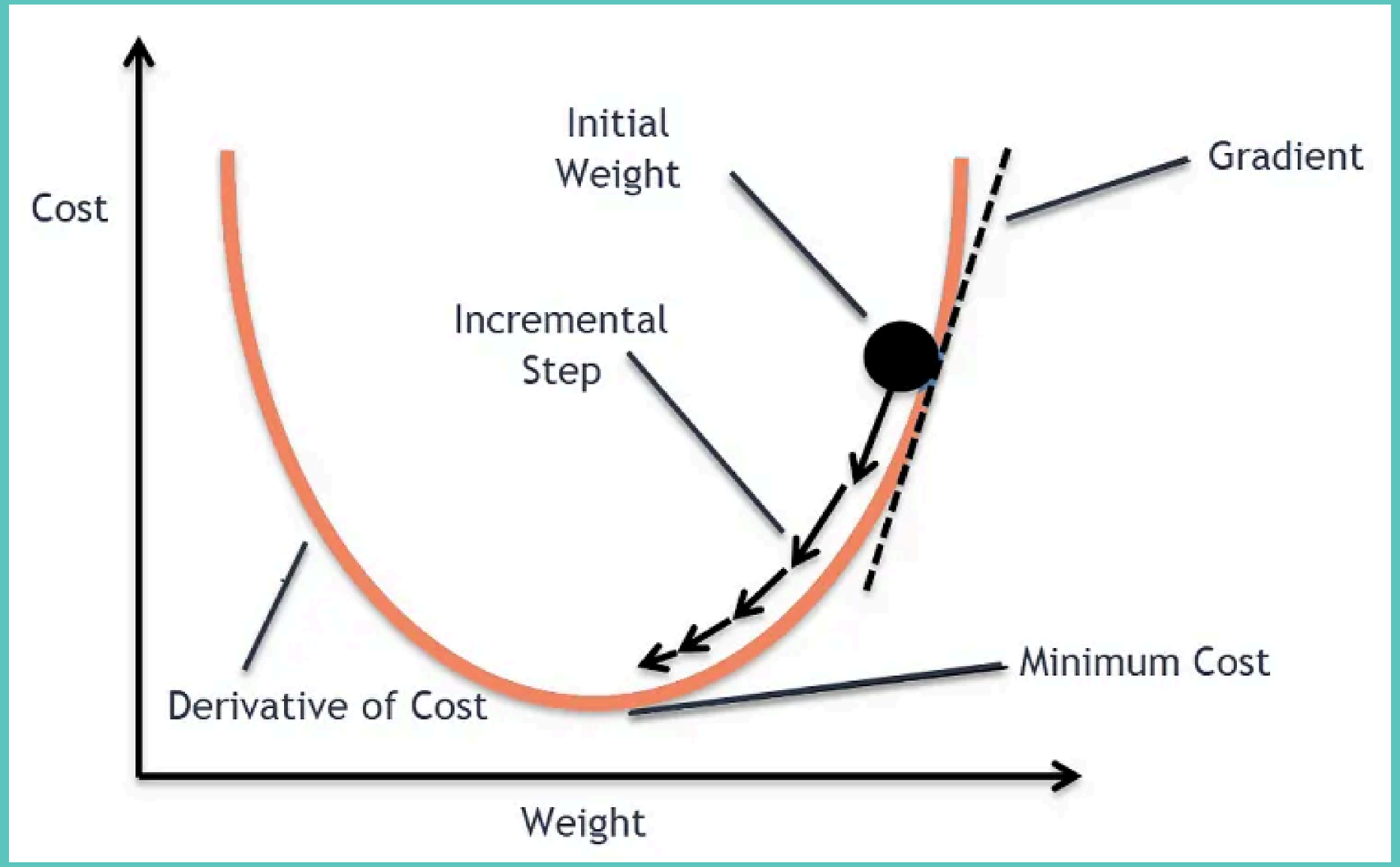
EPOCHS



Updates model parameters

```
for epoch in range(30):
    model.train()
    for cats, nums, y in train_dl:
        cats, nums, y = cats.to(device), nums.to(device), y.to(device)
        preds = model(cats, nums)
        loss  = criterion(preds, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

OPTIMIZER


Cost
Initial
Weight
Gradient
Incremental
Step
Minimum Cost
Derivative of Cost
Weight

EPOCHS

```
epoch 00  train_rmse=28.9461  val_rmse=29.1354
epoch 01  train_rmse=27.9979  val_rmse=27.8641
epoch 02  train_rmse=25.9712  val_rmse=26.2334
epoch 03  train_rmse=24.0906  val_rmse=23.3351
epoch 04  train_rmse=21.9057  val_rmse=20.6031
epoch 05  train_rmse=18.3311  val_rmse=17.7518
epoch 06  train_rmse=14.7346  val_rmse=13.9603
epoch 07  train_rmse=11.1045  val_rmse=11.1646
epoch 08  train_rmse=7.5717  val_rmse=7.1832
epoch 09  train_rmse=3.2365  val_rmse=4.0312
epoch 10  train_rmse=2.5458  val_rmse=3.2084
epoch 11  train_rmse=2.3102  val_rmse=1.3594
epoch 12  train_rmse=2.2729  val_rmse=1.2721
epoch 13  train_rmse=2.3433  val_rmse=1.3703
epoch 14  train_rmse=2.3702  val_rmse=1.4114
epoch 15  train_rmse=2.2201  val_rmse=1.4407
epoch 16  train_rmse=2.2556  val_rmse=1.2102
epoch 17  train_rmse=2.3365  val_rmse=1.1999
epoch 18  train_rmse=2.3754  val_rmse=1.3705
epoch 19  train_rmse=2.3078  val_rmse=1.3034
epoch 20  train_rmse=2.1133  val_rmse=1.2181
epoch 21  train_rmse=2.0834  val_rmse=1.2408
epoch 22  train_rmse=2.7226  val_rmse=1.2248
epoch 23  train_rmse=2.1811  val_rmse=1.1957
epoch 24  train_rmse=2.4305  val_rmse=1.1936
epoch 25  train_rmse=1.9953  val_rmse=1.2015
epoch 26  train_rmse=2.1134  val_rmse=1.2048
epoch 27  train_rmse=2.1425  val_rmse=1.1907
epoch 28  train_rmse=2.1634  val_rmse=1.2768
epoch 29  train_rmse=2.2544  val_rmse=1.2687
```

EPOCHS

```
epoch 00  train_rmse=28.9461  val_rmse=29.1354
epoch 01  train_rmse=27.9979  val_rmse=27.8641
epoch 02  train_rmse=25.9712  val_rmse=26.2334
epoch 03  train_rmse=24.0906  val_rmse=23.3351
epoch 04  train_rmse=21.9057  val_rmse=20.6031
epoch 05  train_rmse=18.3311  val_rmse=17.7518
epoch 06  train_rmse=14.7346  val_rmse=13.9603
epoch 07  train_rmse=11.1045  val_rmse=11.1646
epoch 08  train_rmse=7.5717  val_rmse=7.1832
epoch 09  train_rmse=3.2365  val_rmse=4.0312
epoch 10  train_rmse=2.5458  val_rmse=3.2084
epoch 11  train_rmse=2.3102  val_rmse=1.3594
epoch 12  train_rmse=2.2729  val_rmse=1.2721
epoch 13  train_rmse=2.3433  val_rmse=1.3703
epoch 14  train_rmse=2.3702  val_rmse=1.4114
epoch 15  train_rmse=2.2201  val_rmse=1.4407
epoch 16  train_rmse=2.2556  val_rmse=1.2102
epoch 17  train_rmse=2.3365  val_rmse=1.1999
epoch 18  train_rmse=2.3754  val_rmse=1.3705
epoch 19  train_rmse=2.3078  val_rmse=1.3034
epoch 20  train_rmse=2.1133  val_rmse=1.2181
epoch 21  train_rmse=2.0834  val_rmse=1.2408
epoch 22  train_rmse=2.7226  val_rmse=1.2248
epoch 23  train_rmse=2.1811  val_rmse=1.1957
epoch 24  train_rmse=2.4305  val_rmse=1.1936
epoch 25  train_rmse=1.9953  val_rmse=1.2015
epoch 26  train_rmse=2.1134  val_rmse=1.2048
epoch 27  train_rmse=2.1425  val_rmse=1.1907
epoch 28  train_rmse=2.1634  val_rmse=1.2768
epoch 29  train_rmse=2.2544  val_rmse=1.2687
```

Close but...

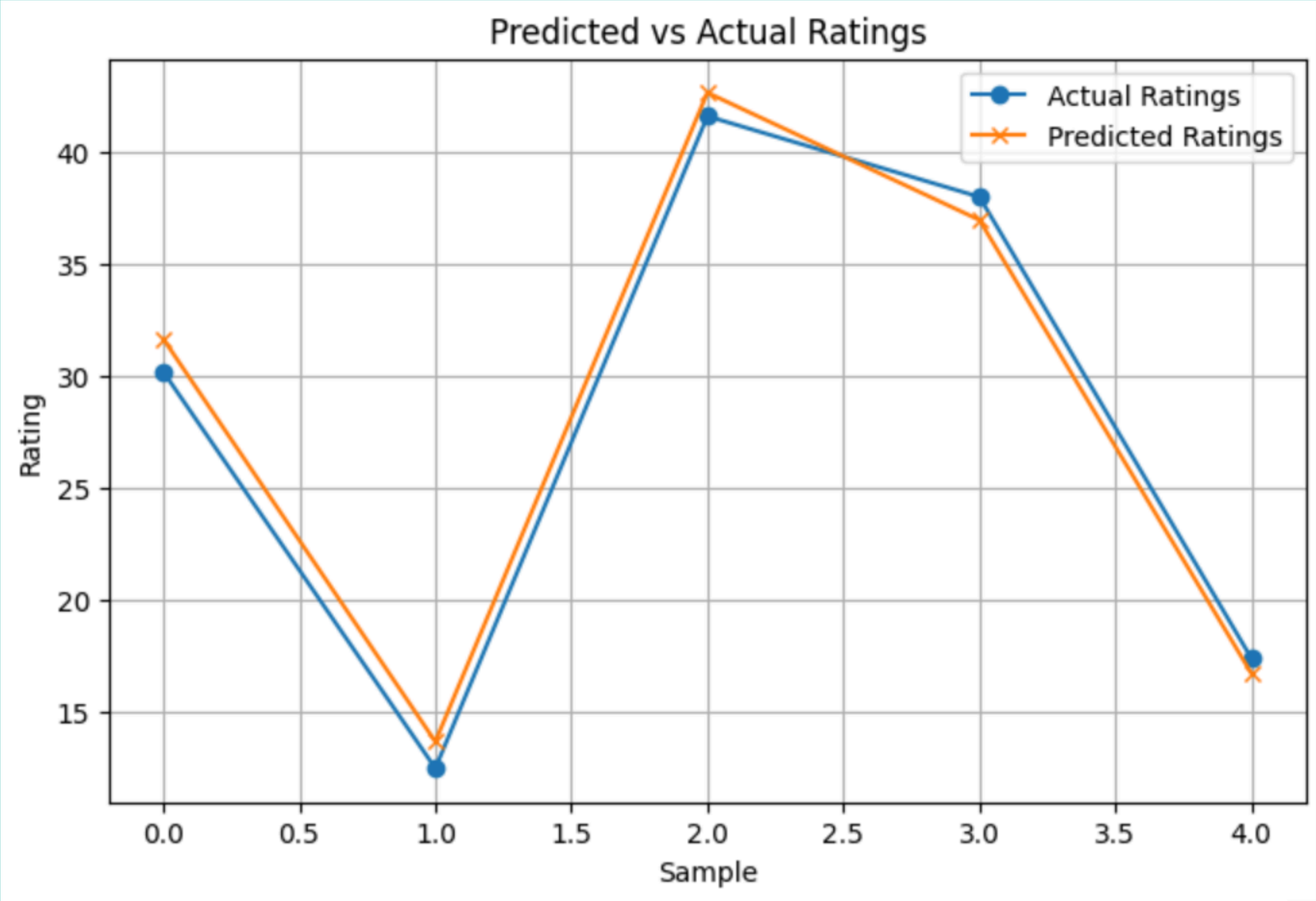
EPOCHS

```
epoch 00 | train_RMSE=29.167 | val_RMSE=29.103
epoch 01 | train_RMSE=27.889 | val_RMSE=27.818
epoch 02 | train_RMSE=26.542 | val_RMSE=26.482
epoch 03 | train_RMSE=23.818 | val_RMSE=23.772
epoch 04 | train_RMSE=21.583 | val_RMSE=21.557
epoch 05 | train_RMSE=18.792 | val_RMSE=18.779
epoch 06 | train_RMSE=16.522 | val_RMSE=16.525
epoch 07 | train_RMSE=13.647 | val_RMSE=13.634
epoch 08 | train_RMSE=11.405 | val_RMSE=11.395
epoch 09 | train_RMSE=9.768 | val_RMSE=9.757
epoch 10 | train_RMSE=7.607 | val_RMSE=7.603
epoch 11 | train_RMSE=5.581 | val_RMSE=5.589
epoch 12 | train_RMSE=4.554 | val_RMSE=4.563
epoch 13 | train_RMSE=2.632 | val_RMSE=2.647
epoch 14 | train_RMSE=2.475 | val_RMSE=2.485
epoch 15 | train_RMSE=2.504 | val_RMSE=2.511
epoch 16 | train_RMSE=1.577 | val_RMSE=1.585
epoch 17 | train_RMSE=1.586 | val_RMSE=1.593
epoch 18 | train_RMSE=1.508 | val_RMSE=1.510
epoch 19 | train_RMSE=1.330 | val_RMSE=1.343
epoch 20 | train_RMSE=1.296 | val_RMSE=1.307
epoch 21 | train_RMSE=1.253 | val_RMSE=1.265
epoch 22 | train_RMSE=1.208 | val_RMSE=1.219
epoch 23 | train_RMSE=1.210 | val_RMSE=1.222
epoch 24 | train_RMSE=1.229 | val_RMSE=1.243
epoch 25 | train_RMSE=1.252 | val_RMSE=1.262
epoch 26 | train_RMSE=1.199 | val_RMSE=1.217
epoch 27 | train_RMSE=1.198 | val_RMSE=1.213
epoch 28 | train_RMSE=1.171 | val_RMSE=1.186
epoch 29 | train_RMSE=1.205 | val_RMSE=1.220
```

4. WHY RMSE?

```
[ ]  class RMSELoss(nn.Module):
         def __init__(self, eps=1e-6):
             super().__init__()
             self.mse = nn.MSELoss()
             self.eps = eps

         def forward(self, yhat, y):
             return torch.sqrt(self.mse(yhat, y) + self.eps)
```

ROOT MEAN SQUARE ERROR VISUALIZE



CONCLUSION

The PyTorch neural network discovered hidden patterns and successfully learned to predict video game user ratings!

Key patterns it captured:

- **Publisher Influence**: Some companies consistently affect ratings.
- **Genre Effects**: Certain genres trend higher or lower.
- **Release Year Trends**: Older and newer games show distinct patterns.
- **Other Features**: Price, game length, and quality scores (graphics, soundtrack, story) matter too.

Final results:

- **Training RMSE**: ~1.2
- **Validation RMSE**: ~1.2
  - (Starting from RMSE > 28!)

Training involved:

- **Data Cleaning**: Removing irrelevant fields.
- **Feature Engineering**: Organizing numerical and categorical inputs.
- **Embedding Layers**: Efficiently encoding large categories.
- **Model Training**: 30 epochs with steady improvement.
- **Loss Function**: RMSE for clear, interpretable accuracy.

A custom **RMSELoss** class **stabilized** training, and the **Predicted** vs. **Actual** plot confirmed tight, **consistent predictions**.